

Engineering Journal (Log)

Team : Sleeping

Participating unit: Macau Pui Ching Middle School

Team member:

Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Date of creation: 14th February,2026

Table of contents

Operational Logistic.....	3
Log number: 1 Event date: 14th of February,2026.....	4
Log number: 2 Event date: 18th of February,2026.....	6
Log number: 3 Event date: 21th of February,2026.....	9
Log number: 4 Event date: 27th of February,2026.....	10
Log number: 5 Event date: 5th of March,2026.....	14
Log number: 6 Event date: 10th of March,2026.....	17
Log number: 7 Event date: 15th of March,2026.....	21
Log number: 8 Event date: 21th of March,2026.....	24
Log number: 9 Event date: 27th of March,2026.....	27
Log number: 10 Event date: 1st of April,2026.....	29
Log number: 11 Event date: 5th of April,2026.....	35
Log number: 12 Event date: 11th of April,2026.....	39
Log number: 13 Event date: 11th of April,2026.....	40

Operational Logistic

- Means of communication:
 - **Verbal communication:** Our team can communicate verbally and discuss ideas, opinions, and facts by having conversations between members. It is among the easiest and most efficient methods for several people to work together productively in a group. Each member of our team is tasked with Overseeing several project areas through verbal communication.
- Meeting frequency:
 - We as a team agreed to have three weekly meetings. Meeting every day would make time more compact because we still have to go to school. If we had to maintain our concentration on both our project and our academic responsibilities, we would not be able to guarantee our productivity. As a result, our team thinks that meeting three times a week is the most effective way to handle our workload.
- Location:
 - **Macau Pul Ching Secondary School Innovation Science and Education Center (Fablab):** We are able to access a wide range of components in the laboratory because of the diversity of supplements. Not to add, Fablab offers an adequate and comfortable environment for us to debug our program and robot. In order to maintain the integrity of the work, educators working in the laboratory can also offer suggestions for our project.
- Distribution of work:
 - Assign work to members, create personal task: lists and attendance calendars, and make sure that everyone completes the assigned duties in the allotted time Here is a table with the task assignments:

Members	Robot designs	Hardware	Programming	Log book
Li Cheng Lok	✓	✓	✓	✓
Lloyd	✓			✓
Mike.V			✓	✓
Wong loi Cheng		✓		✓

Log number: 1

Event date: 14th of February,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

- Our team documented the robot's limited design and drew the next stage
- comprehending the competition guidelines and test location

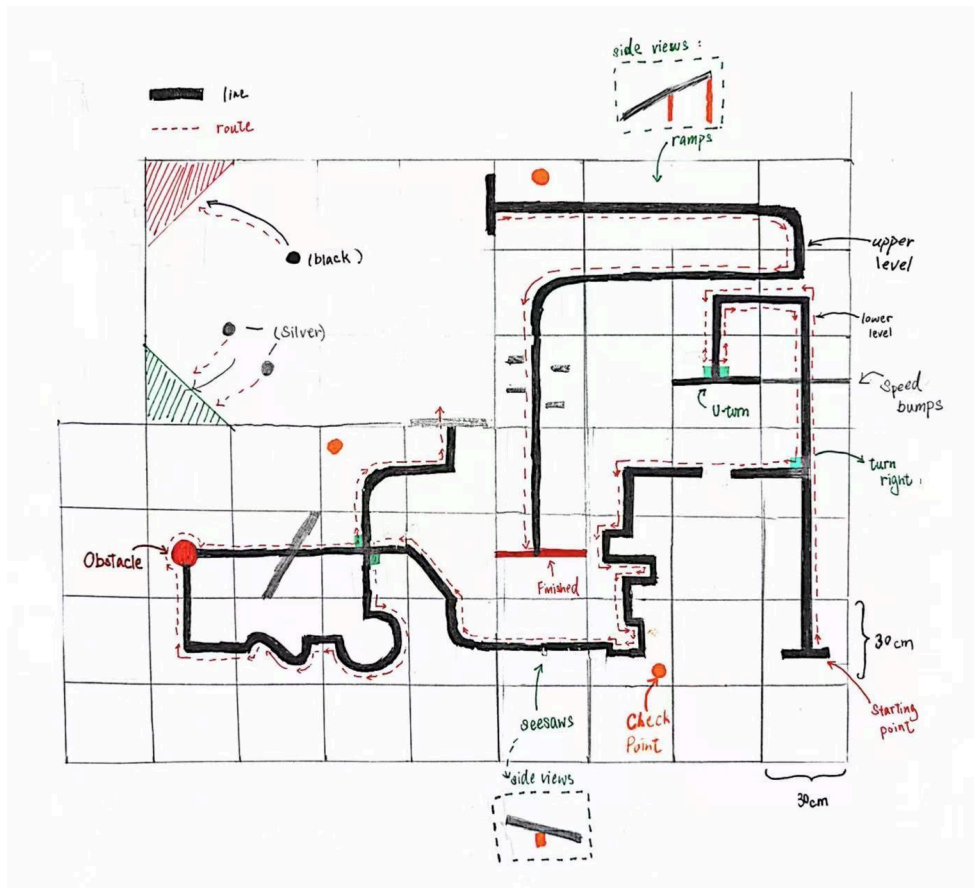
2. Rules Analyse

1. The robot car will need to **start** from the tile with a **black horizontal line**, then follow the line and **stop** on the tile with a **red line**.
2. Different tiles have **different patterns**, and the path may change between **different rounds**.
3. Checkpoints are included in the path, and the robot can **restart** at the tile with checkpoint.
4. **Speed Bumps, Debris, and Obstacles** may appear in the path and robots need to try to come across those.
5. The robot needs to turn according to the direction with **green intersection markers**.
6. Tiles will be used as **ramps** to allow the robot to **climb** up and down from different levels.
7. Robots need to come over and keep balance when crossing a **seesaw**.
8. In the Evacuation Zone, robots need to place the **black** dead victims in the **red evacuation point** and **silver** living victims in the **green evacuation point**.

Our team tries to draw out an example path and try to figure out the route that we should follow, so that we can think about how the robot can come over each ordeal.

Example path:

The red dotted line next to the path is the route that we should follow.



3. Planning

- Complete the robot's initial design according to the require and the need to let the robot be able to come across all the challenges.
- Begin to talk about the coding and algorithm planning that has to be done

Log number: 2

Event date: 18th of February,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

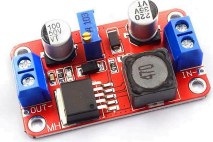
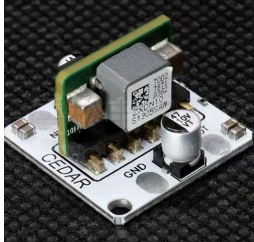
Location: Macau Pui Ching Secondary School Innovation Science and Education Center

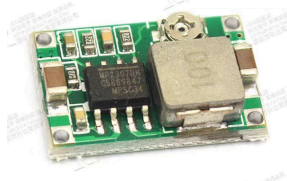
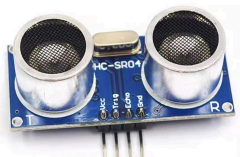
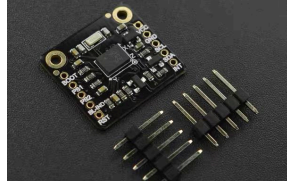
1. Tasks done today

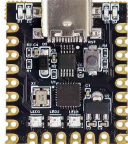
- Finish choosing what sensor or hardware that we are going to use on the robot according to their function and also the competition roles.

2. Hardware introduction

After careful analysis and judgment by our team through the competition rules, we finally selected different sensors to solve different tasks. For example, we chose to use HC-SR04 ultrasonic sensor for getting the distance between robot and obstacles or walls, and we may use USB camera to capture the environment so that the robot can follow the line or catch the ball.

	Hardware	Function	Sensor's picture
1.	DC step up converter	Increase the voltage	
2.	5v step down converter	Change the voltage to 5V	

3.	USB Camera	Capture image in front of the robot	
4.	Picamera 3 wide	Capture image of the line under the robot	
5.	Google coral edge tpu	Provide 4Tops AI performance for raspberry pi	
6.	5v converter	Change the voltage to 5V	
7.	HC-SR04 ultrasonic sensor	Get the distance between robot and obstacles or walls	
8.	emax servo	Control the open and close of the clip	
9.	BNO055	Get the inclination of the robot	

10.	Raspberry pi 5	Main controller	
11.	18650 battery	Power the raspberry pi	
12.	Arduino nano mini	Control the servo	
13.	Micro switch	Detecting obstacles	
14.	v153l0x	get distance of the front	

3. Planning

- After selecting the sensor, start designing the robot through 3D design, and design the robot based on the functions of the selected sensor.
- Start trying out a basic walking program, which allows the robot to follow a route.

Log number: 3

Event date: 21th of February,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

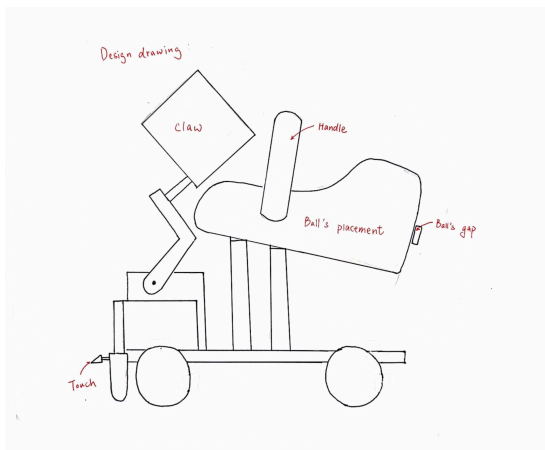
Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

- Start 3D designing the chassis of the robot.

2. Design drawing



3. Planning

- Design the full appearance of the robot based on the needs of the sensors we selected last time.

Log number: 4

Event date: 27th of February,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

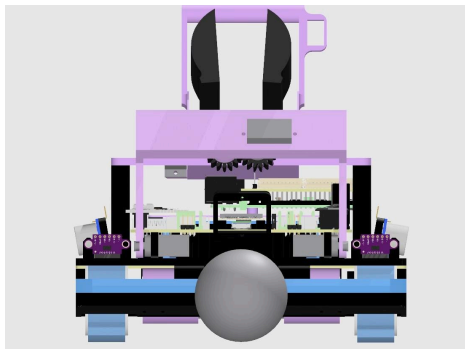
Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

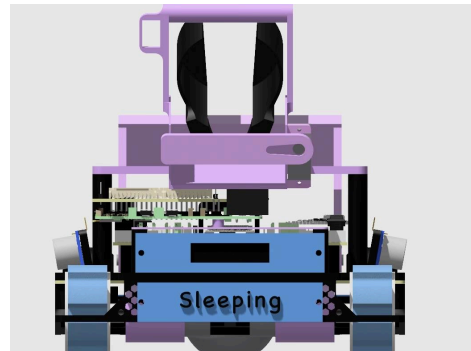
- Design the appearance of the robot based on the needs of the sensors we selected last time by using Shapr3D.

2. Preliminary design

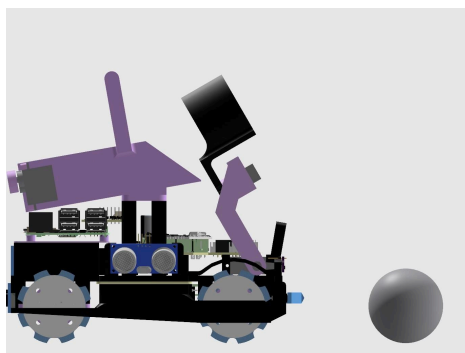
Front:



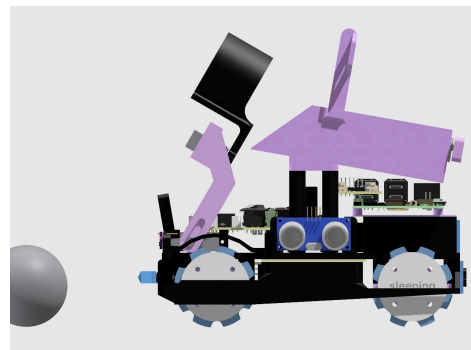
Back:



Right:



Left:



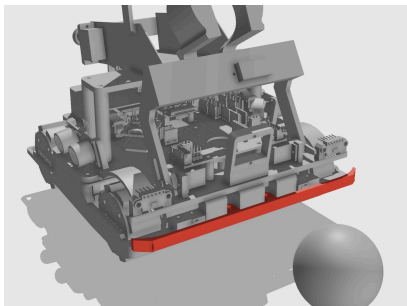
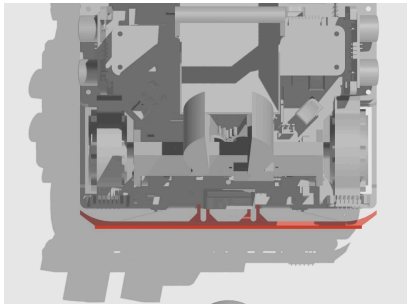
3. Designs detail introduction

Some of the designs you see are tailor-made specifically for this competition, crafted to align with its unique requirements and themes, and will be showcased in detail below.

1) Touch

Function : Detect obstacles while following the line, if it reaches a obstacle it will get around it.

Displayment:

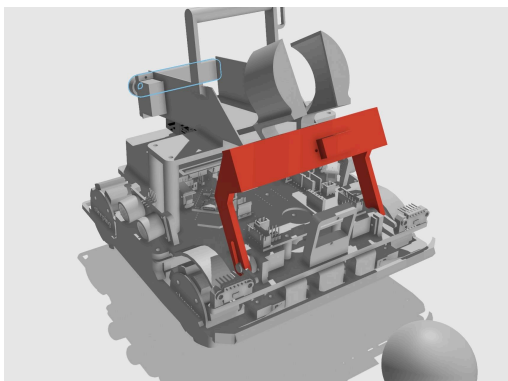


2) Claw

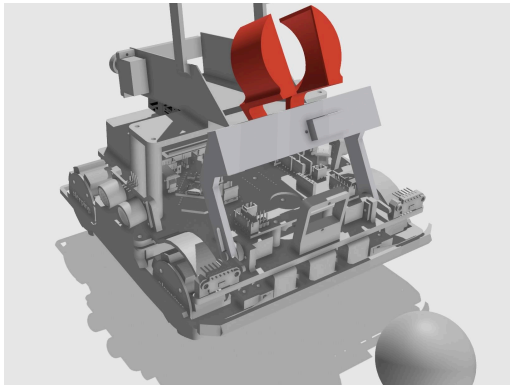
Function : Catch the ball with servos and gears

Displayment:

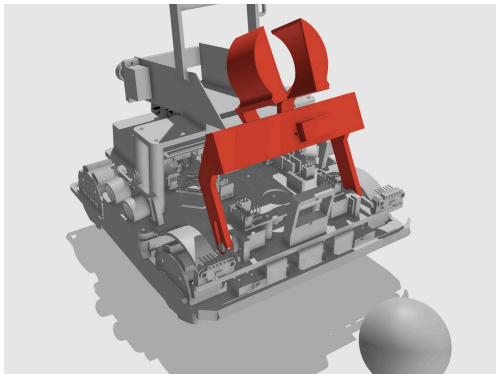
Design that allow the claw to move forward and backward :



Design that allow the claw to open and close :



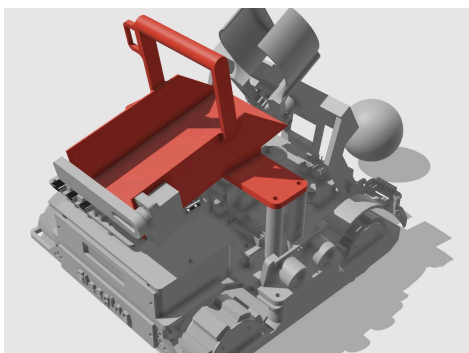
The full design of the claw:



3) Ball's placement

Function : A place where the ball is temporarily stored before reaching the Evacuation Zone and after collecting the ball by the claw. And it is designed with handle.

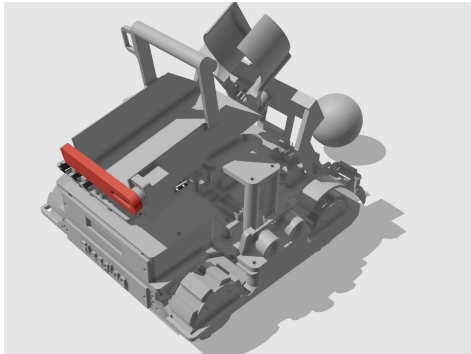
Displayment :



4) Ball's gap

Function : Prevent the ball from rolling down directly. When it reaches the Evacuation Zone, the loop will open and the ball will roll down.

Displayment:



3.Planning

- Start to figure out a basic walking program, which allows the robot to follow a route.

Log number: 5

Event date: 5th of March,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Wong loi Cheng

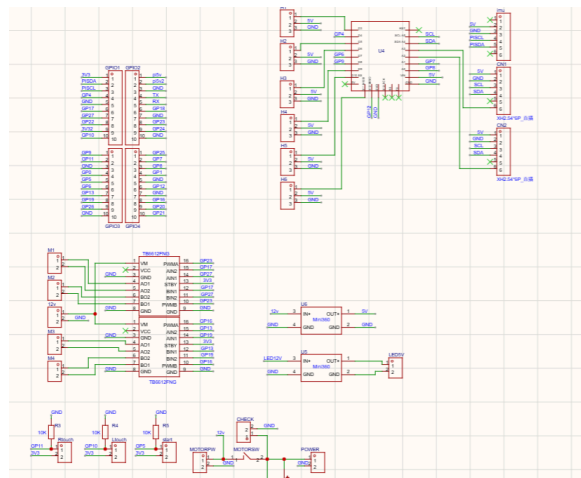
Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1.Tasks done today

- Finish the design of the PCB1 and PCB2,so that electronic parts stay in place and connect properly, a PCB is used in all electronic devices.
- Write the program of collecting the data of the black line.

2.Basic Program

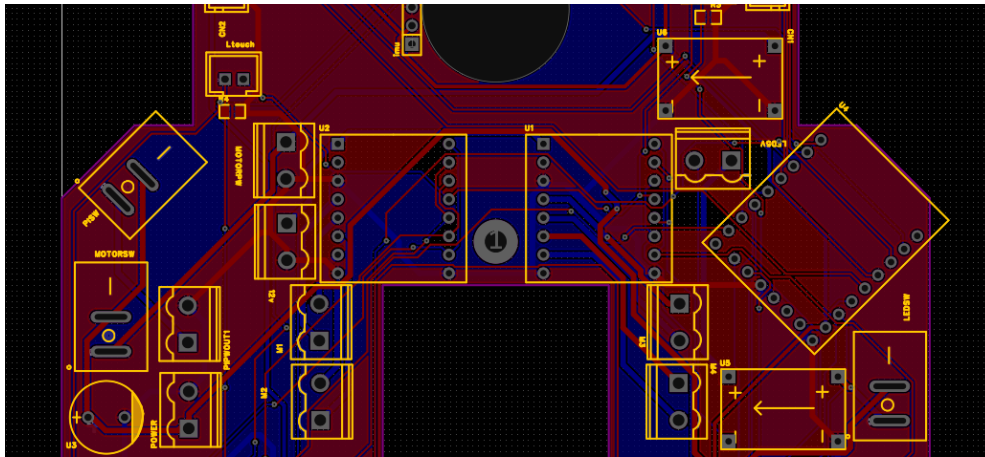
- This is the schematic diagram of PCB 1.



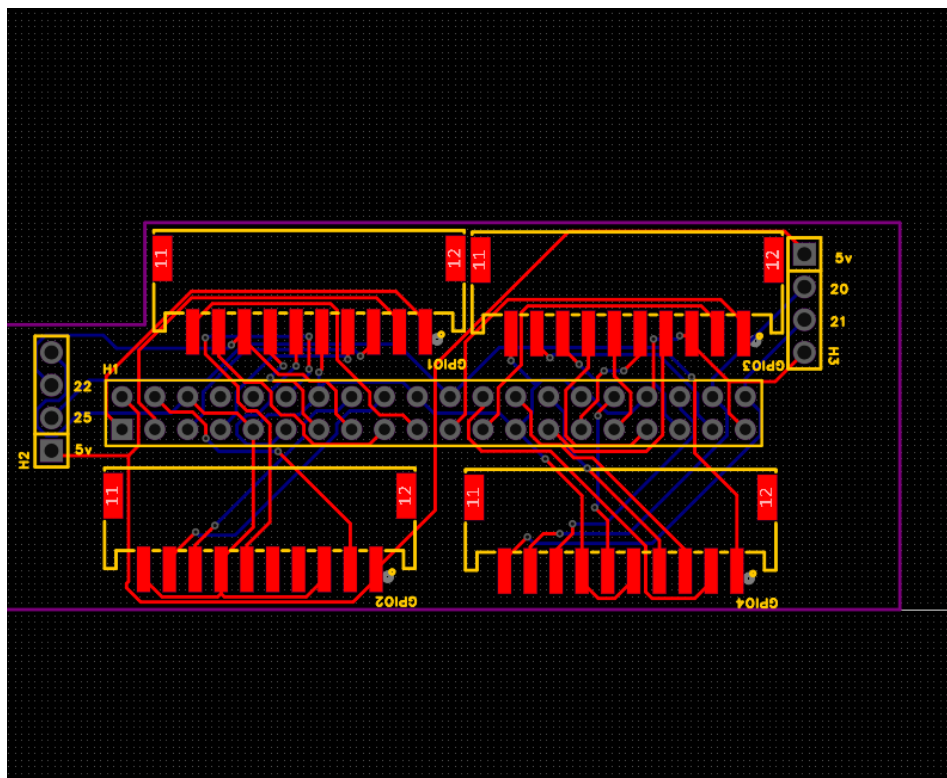
- This is the schematic diagram of PCB 2.



- This is PCB 1



- This is PCB 2



- This is the program of collecting data of lines.

```

1  data_collect.py
2  import cv2
3  import numpy as np
4  from picamera2 import Picamera2
5  from time import *
6  import rcjlib.colorfix as cf
7  from gpiozero import *
8  import os
9
10 button = InputDevice(5)
11 path = r"/home/lok/Desktop/3_17line/line"
12 cam = Picamera2()
13 cam.configure(cam.create_video_configuration(main={"format": 'RGB888', "size": (1920, 1080)}))
14 cam.set_controls({"ExposureTime": 100, "Contrast": 1.4, "Brightness": 0, "Sharpness": 1.5, "Saturation": 1, "ExposureValue": 0.1}) #contrast 0-32 brightness +-1 sharpness 0-16 saturation 0-100
15 cam.start()
16 lut_r = cf.generate_lut(0, 255, 0, 255)
17 lut_g = cf.generate_lut(0, 255, 0, 255)
18 lut_b = cf.generate_lut(0, 255, 0, 240)
19 i = 3234
20 os.chdir(path)
21 while(True):
22     frame = cam.capture_array("main")
23     frame = cv2.resize(frame, (320, 240))
24     frame = frame[10:, 30:240]
25     frame = cv2.resize(frame, (320, 240))
26     b, g, r = cv2.split(frame)
27     r = cv2.LUT(r, lut_r)
28     g = cv2.LUT(g, lut_g)
29     b = cv2.LUT(b, lut_b)
30     frame = cv2.merge((b, g, r))
31     cv2.imshow("cam", frame)
32     kc = cv2.waitKey(1)
33     if kc & 0xFF == ord('q'):
34         break
35     if button.value == 1:
36         cv2.imwrite(f"{i}.jpg", frame)
37         i+=1
38         print("saved", i)
39         sleep(0.01)
40 cv2.destroyAllWindows()

```

3.Planning

- Write the program of following the black line and detect the green intersection marker for turning to different directions.

Log number: 6

Event date: 10th of March,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

- Write the program of following the black line and detect the green intersection marker for turning to different directions.
- Test the program.

2. Basic Program

1) Line following program

- This part of the program divides the captured image into 24 horizontal slices and analyzes the position of the black line within each slice to determine its location.

```
def extract_polygon(img, slice_num=24):
    IMG_HEIGHT, IMG_WIDTH = img.shape
    X_DIV = int(IMG_HEIGHT/float(slice_num))
    kernelOpen = np.ones((5,5))
    kernelClose = np.ones((20,20))
    poly_points = [None] * slice_num
    detected_contours = [None] * slice_num

    for i in range(0, slice_num):
        sliced_img = img[X_DIV*i + 1:X_DIV*(i+1), int(0):int(IMG_WIDTH)]
        maskOpen = cv2.morphologyEx(sliced_img, cv2.MORPH_OPEN, kernelOpen)
        maskClose = cv2.morphologyEx(maskOpen, cv2.MORPH_CLOSE, kernelClose)
        maskFinal = maskClose
        conts, _ = cv2.findContours(maskFinal.copy(), \
                                   cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)

        if(len(conts)):
            c = max(conts, key = cv2.contourArea)

            M = cv2.moments(c)
            poly_points[i] = (int(M['m10']/M['m00']), int(M['m01']/M['m00']) + X_DIV * i)
            detected_contours[i] = c

    contours = [i for i in detected_contours if i is not None]
    points = [i for i in poly_points if i is not None]
    return points, contours

def evaluate_function(angle_part, translate_part, x, y):
    x = abs(x)
    return math.exp(-1*y/240/20.)*(
        (1 - 1.*x/320/2.0)*angle_part +
        .8*x/320 * translate_part)
```

- By extracting contours and calculating moments, the program identifies the center point of the black line in each segment, allowing the robot to adjust its path

accordingly.

- The program can detect the position of the black line that is closest to the edge of the image, ensuring accurate tracking even when the line is near the boundary.

```
class FollowState(State):
    def on_switch(self, sm, old_state_name):
        global obsdir
        if obsdir == 0:
            obsdir = 1
        else:
            obsdir = 0
    def __init__(self):
        self.goingup = False

    def calc_ctrl(self, points):
        self.edgepointlist = []
        self.pointx = 0
        self.ang = 0
        self.Lineang = 0
        self.min = 14
        self.edgepointy = 8
        self.edgepoint = [160, 0]
        if len(points) == 0:
            self.ang = 0
            self.Lineang = 0
            self.Line = 0
        else:
            y = points[:, 1]
            x = points[:, 0].astype(np.float64)
            #x[x > 240] ^= 1.1
            x = 160 - np.abs(x - 160)

            idx = np.argmin(np.min(np.array([x, y]), axis=0))
            self.edgepoint = list(points[idx])[0][1], (x2, y2)]

            self.Lineang = math.degrees(math.atan((points[0][0] - points[-1][0]) / ((240 - points[0][1]) - (120 - points[-1][1]))))
            self.Line = (self.edgepoint[0]-160) * (points[0][0] - points[-1][0])

            self.Linedistance = math.sqrt((self.edgepoint[0]-160)**2 + (self.edgepoint[1]-120)**2)

            self.ctrl = controller.control(self.Line, 0)
            print(self.Line, self.ang, self.Lineang)

    def check_noise(self, points):
        hull = cv2.convexHull(points)
        area = cv2.contourArea(hull)

        #print(area)
        if area > 22000:
            print(area)
            return True
        else:
            return False
```

1) Green intersection marker detecting program

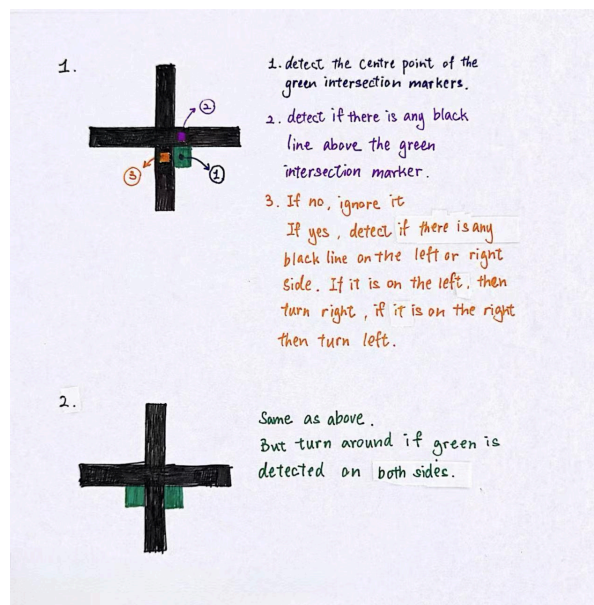
- This part of the program is designed to detect green intersection markers on the track, which are used to control the robot's turning direction at junctions.
- It uses color segmentation in the HSV space to isolate green regions, filters out noise by size and shape, and identifies valid markers for decision-making.
- Based on the position of the green marker and the surrounding black lines, the program determines whether the robot should turn left, turn right, or go straight.

```

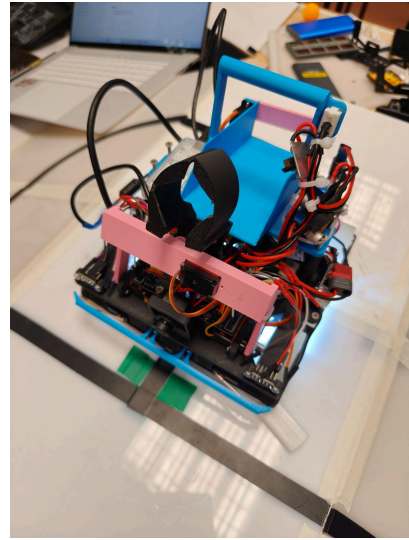
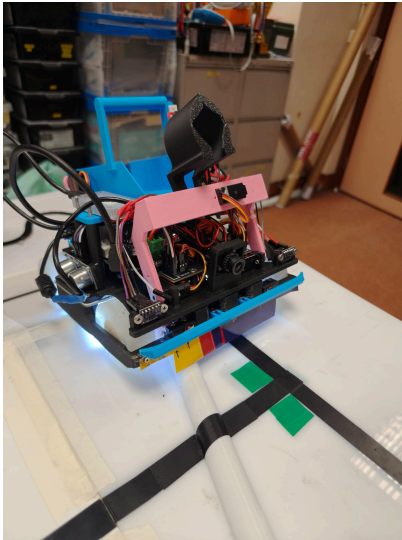
def detect_green_boxes(frame, gray, display_buffer, size=60):
    green_lower = np.array([0,40,0])
    green_upper = np.array([100,255,100])
    mask = cv2.inRange(frame, green_lower, green_upper)
    mask = cv2.morphologyEx(mask, cv2.MORPH_DILATE,
        kernel=np.ones((5,5), np.uint8), iterations=3)
    mask = cv2.dilate(mask, kernel, iterations=3)
    mask = cv2.erode(mask, kernel, iterations=3)
    cv2.imshow('mask', mask)
    contours, _ = cv2.findContours(mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    green_x = []
    green_y = []
    good_green_index = []
    for contour in contours:
        x, y, M, m = cv2.boundingRect(contour)
        print(x, y)
        if w > 25 and h > 40:
            center_x, center_y = x + w // 2, y + h // 2
            green_x.append(center_x)
            green_y.append(center_y)
            #display_buffer = cv2.circle(display_buffer, (center_x, center_y), 10, (0,0,255), -1)
    if len(green_y) == 0 or min(green_y) < 70:
        return False, False
    for i, (x, y) in enumerate(zip(green_x, green_y)):
        if y < size // 2 + 40:
            continue
        if y > 150:
            continue
        if x > 310 or x < 10:
            continue
        upper_area = np.sum(gray[
            y - size // 2 - 40:y - size // 2 - 20,
            x - 10:x + 10, ])
        display_buffer = cv2.rectangle(display_buffer, (x - 10, y - size // 2 - 20), (x + 10, y - size // 2 - 40), (128, 128, 128), 3)
        #print(upper_area)
        if upper_area / 400 < 70:
            good_green_index.append(i)
        display_buffer = cv2.circle(display_buffer, (x,y), 10, (255,0,0), -1)
    #print(len(good_green_index))
    if len(good_green_index) >= 2: return True, True
    if len(good_green_index) == 0: return False, False
    x = green_x[good_green_index[0]]
    y = green_y[good_green_index[0]]
    if x < size // 2 + 40: return True, False
    left_area = np.sum(gray[
        y-10:y+10, x-size // 2 - 40: x-size // 2 - 20
    ])
    frame = cv2.rectangle(frame, (x - size // 2 - 40, y - 10), (x - size // 2 - 20, y + 10), (128, 128, 128), 3)
    if left_area / 400 > 70: return True, False
    return False, True

```

Here is the manuscript of the detecting order and how to determine which direction to go. And also how it works when the robot need to turn around.



3. Actual measurement



4. Planning

- Take photos of the balls and collect data.
- Train the model of identifying the balls.

Log number: 7

Event date: 15th of March,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

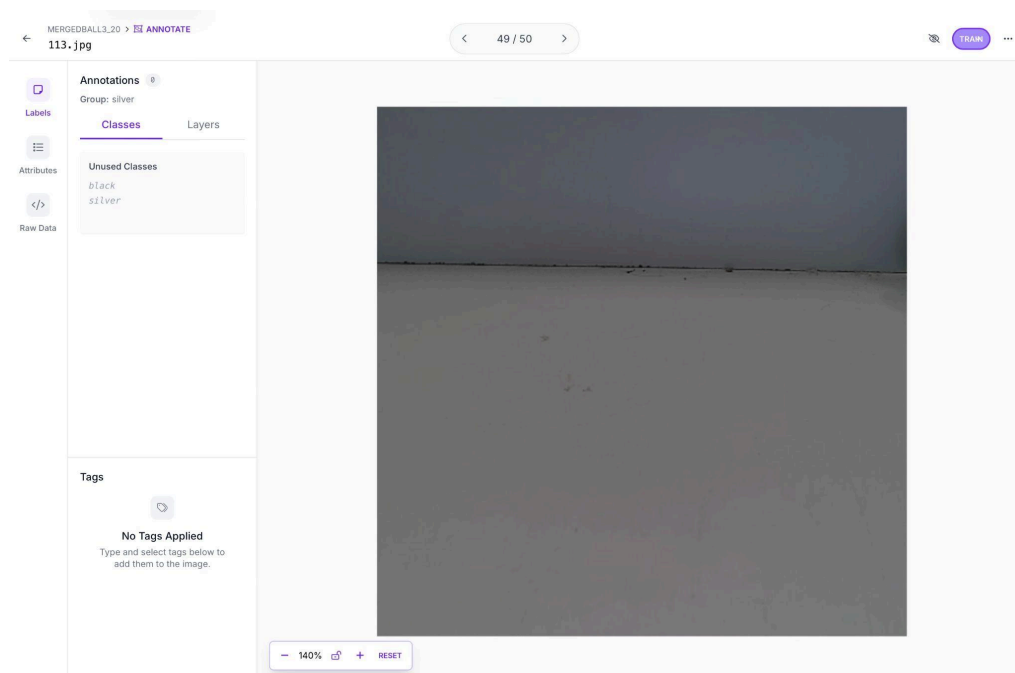
- Take photos of the balls and collect data.
- Train the model of identifying the balls.

2. Photos

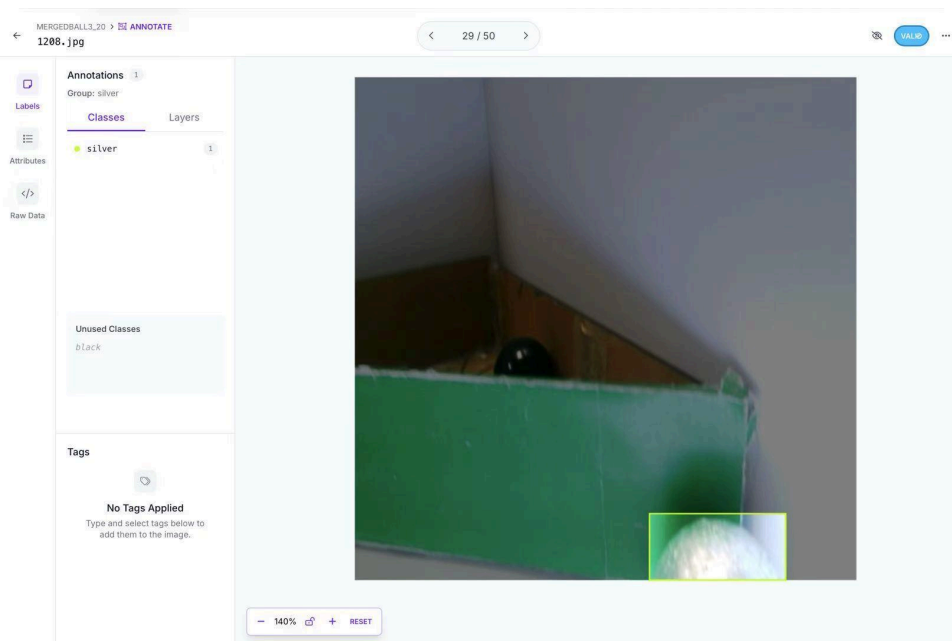
These are the photos examples for collecting data. We have collected more than 5,000 photos from different angles to detect balls of different colors, so that the ball collection process can be faster and more accurate.

In this section, we used universe.roboflow.com to train the model.

This is a photo example with no ball, and it will be marked as no annotation.

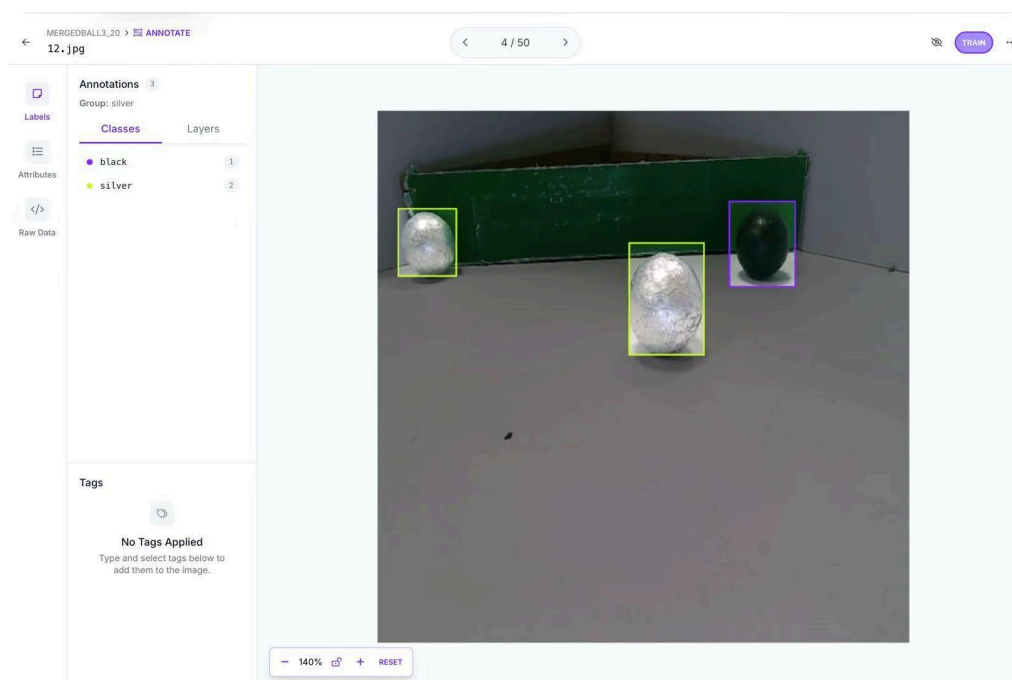


Balls that are in the red or green evacuation point won't be detected.

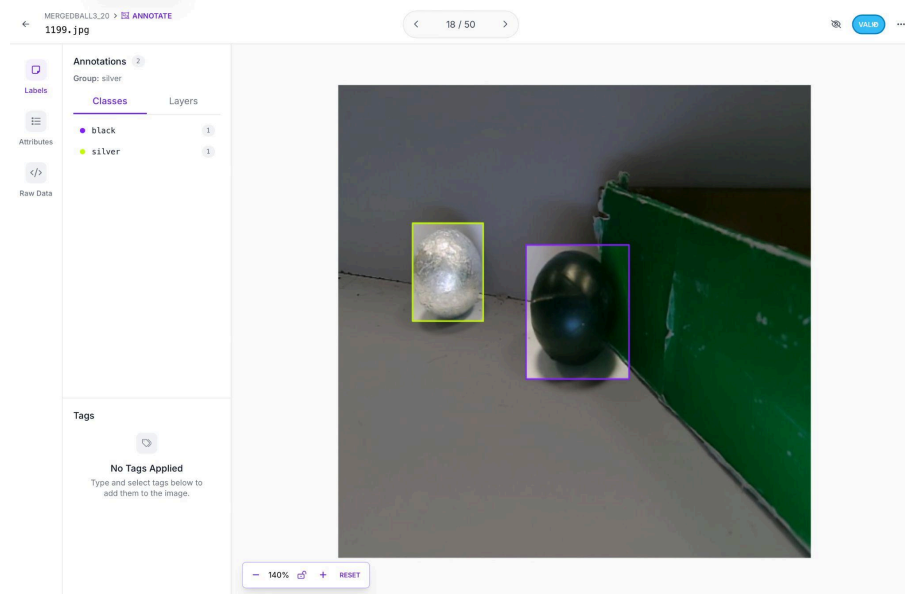


Here is examples of detecting balls, silver balls is marked with a green block, and black ball is marked with a purple block, more than one ball can be marked at the same time.

Example 1.



Example 2.



4.Planning

- Start to train the robot to overcome challenges like getting around obstacles,etc.
- Detect the red or green evacuation.

Log number: 8

Event date: 21th of March,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd, Mike.V

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

- Start to train the robot to overcome challenges like getting around obstacles,etc.
- Detect the red or green evacuation.

2. Program

These are the programs for obstacle avoidance.

When touch detects obstacles:

```
if L.value == 1 and R.value == 1:  
    return sm.switch("Line_Obstacle")
```

Get around obstacles:

```
class ObstacleState(State):  
    def on_tick(self, sm: StateMachine, framegray, *args, **kwargs):  
        global obsdir  
        if L.value == 1:  
            chassis.leftmotor(40)  
            chassis.rightmotor(-100)  
        if R.value == 1:  
            chassis.leftmotor(-100)  
            chassis.rightmotor(40)  
        if sm.get_elapsed_time() <= 0.2:  
            chassis.leftmotor(-60)  
            chassis.rightmotor(-60)  
        elif sm.get_elapsed_time() <= 1.1:  
            if obsdir == 0:  
                chassis.leftmotor(80)  
                chassis.rightmotor(-80)  
            else:  
                chassis.leftmotor(-80)  
                chassis.rightmotor(80)  
        elif sm.get_elapsed_time() <= 1.4:  
            chassis.leftmotor(100)  
            chassis.rightmotor(100)  
        elif sm.get_elapsed_tick() % 10 <= 3:  
            chassis.leftmotor(80)  
            chassis.rightmotor(80)
```

```

else:
    if obsdir == 0:
        chassis.leftmotor(-60)
        chassis.rightmotor(80)
    else:
        chassis.leftmotor(80)
        chassis.rightmotor(-60)

    avg_pix = np.sum(framegray[
        CAM_HEIGHT//2 - 25:CAM_HEIGHT//2 + 25,
        CAM_WIDTH//2 - 25: CAM_WIDTH//2 + 25]) / 5000
    #print(avg_pix)
    if sm.get_elapsed_time() > 2:
        if avg_pix < 50:
            if obsdir == 0:
                return sm.switch("Obs_Right")
            else:
                return sm.switch("Obs_Left")

```

"Return to route" after obstacle avoidance is completed :

```

class OBSLeftTurnState(State):
    def on_tick(self, sm: StateMachine, *args, **kwargs):

        if sm.get_elapsed_time() <= 1.3:
            chassis.leftmotor(-40)
            chassis.rightmotor(80)
        elif sm.get_elapsed_time() <= 1.4:
            chassis.leftmotor(-80)
            chassis.rightmotor(-100)
        else:
            return sm.switch("Line_Follow")

class OBSRightTurnState(State):
    def on_tick(self, sm: StateMachine, *args, **kwargs):

        if sm.get_elapsed_time() <= 1.3:
            chassis.leftmotor(80)
            chassis.rightmotor(-40)
        elif sm.get_elapsed_time() <= 1.4:
            chassis.leftmotor(-100)
            chassis.rightmotor(-80)
        else:
            return sm.switch("Line_Follow")

```

3.Red or green evacuation

Here are the programs of red or green evacuation.

This code defines the `FindPutState` class, which detects either a green or red drop-off zone based on the given color parameter. It waits 0.6 seconds after entering the state for stable detection, then creates a color mask to isolate the target zone, displays the mask, and finds contours in the masked image to locate the drop-off area.

```
class FindPutState(State):
    def __init__(self, color = None):
        self.color = color
        self.have = False

    def on_tick(self, sm: StateMachine, ballframe, *args, **kwargs):
        global putted
        if sm.get_elapsed_time() > 0.6:
            xs = []
            ws = []
            x = 320
            if self.color == 1:
                #green
                lower = np.array([0, 40,0])
                upper = np.array([90, 255, 50])
            elif self.color == 0:
                #red
                lower = np.array([0, 0,70])
                upper = np.array([90, 90, 255])
            mask = cv2.inRange(ballframe,lower, upper)
            #mask = cv2.dilate(mask, None, iterations=2)
            # mask = cv2.erode(mask, None, iterations=5)
            # mask = cv2.dilate(mask, None, iterations=12)
            cv2.imshow("mask",mask)
            contours, _ = cv2.findContours(mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
```

4.Planning

- create a venue for robot training.

Log number: 9

Event date: 27th of March,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

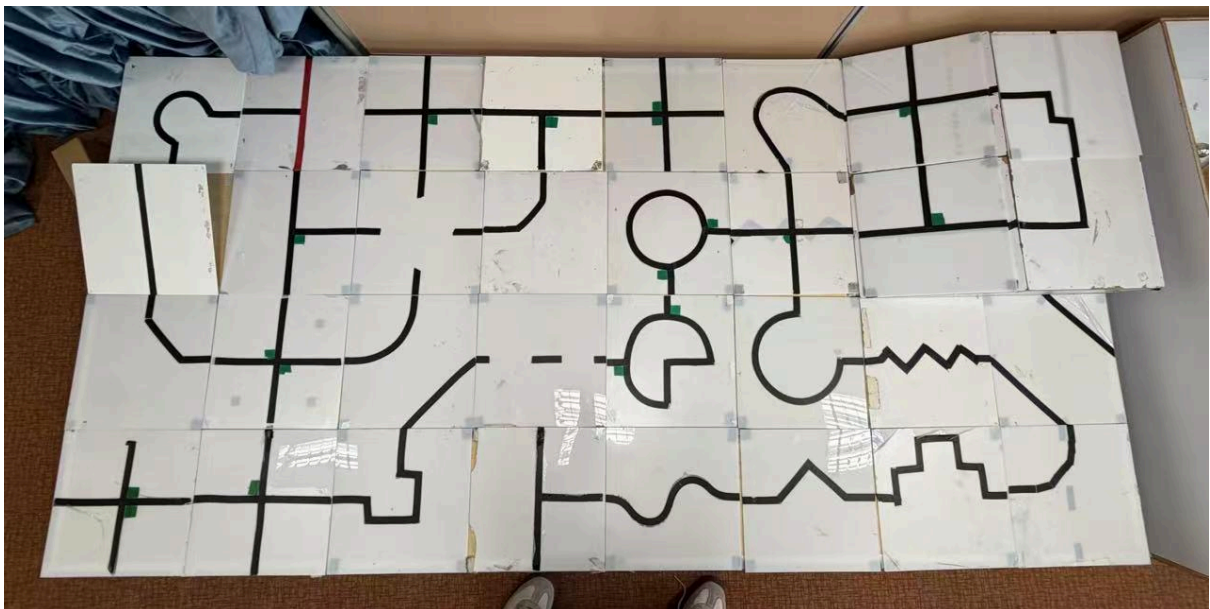
1. Tasks done today

- create a venue for robot training.

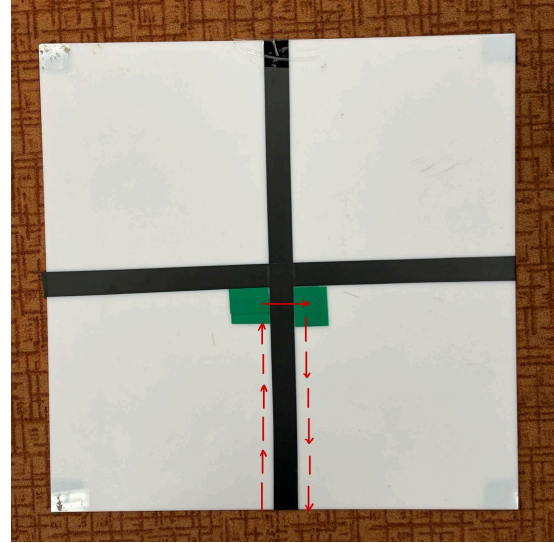
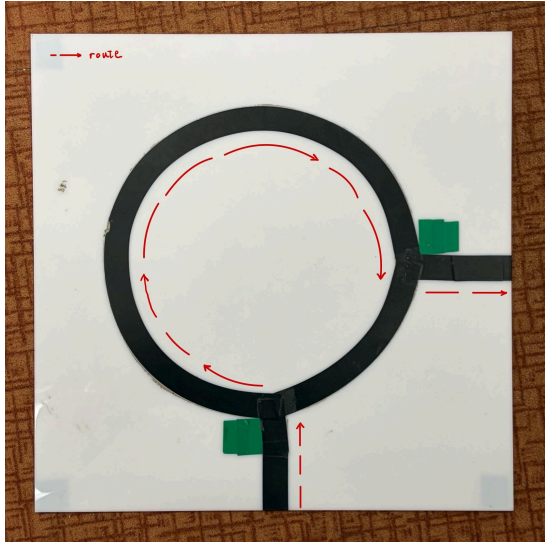
2. Photos

We use tape to post different patterns to meet our daily training and make the robot more stable.

The whole venue :



Some examples (the red dotted line is the route):



3.Planning

- complete the ball part after entering the room.

Log number: 10

Event date: 1st of April,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

- complete the ball part after entering the room.

2. Program

Included:

- Seek ball
- Detection ball (YOLO)
- Pick up the ball
- Find the putting area
- Release the ball
- Counting the number of balls picked up and the number of balls released

1) Variable records the number of balls picked up and released

```
picked = 0  
putted = 0
```

2) YOLO model detection ball

```
ballmodel = YOLO(r"model/3_20ball_full_integer_quant_edgetpu.tflite", task="detect")
```

3) FindBallState — find and move toward the ball

```

class FindBallState(State):
    def __init__(self):
        self.color = 1

    def detectballs(self, ballframe, color):
        results = ballmodel.predict(ballframe, imgsz = 320)
        ballframe = results[0].plot()
        #cv2.imshow("Ball", self.ball_frame)
        # Visualize the results on the frame

        boxes = results[0].boxes

        for box in boxes:
            #print(box.cls)
            cx, cy, w, h = box.xywh[0]
            if int(box.cls) == color: #black-0 silver-1
                return cx,cy, w, h, ballframe
            else:
                return -1,-1, -1, -1, ballframe
        return -1,-1, -1, -1, ballframe
    def on_switch(self, sm, old_state_name):
        self.laststate = old_state_name

```

```

def on_tick(self, sm: StateMachine, *args, **kwargs):
    global picked, ballframe, putted
    #print(self.laststate,picked)
    #print(putted, picked)
    if sm.get_elapsed_time() > 0.7:
        if putted == 2:
            self.color = 0
        else:
            self.color = 1
        if putted == 3:
            chassis.leftmotor(0)
            chassis.rightmotor(0)
            return sm.switch("BGo")
        elif picked == 3:
            return sm.switch("FindPut", params=0)

        if picked == 2 and putted == 0:
            return sm.switch("FindPut", params=self.color)

```

```

cx,cy, w, h, ballframe = self.detectballs(ballframe, self.color)
#print(cx,cy,w,h)
if not(cx == -1 and cy == -1):
    if cx > 140 and cx < 180:
        if cy > 200:
            return sm.switch("LineGoBack")
        elif cy > 175:
            chassis.leftmotor(0)
            chassis.rightmotor(0)
            picked += 1
            return sm.switch("Pick")
        elif cy < 80:
            error = 160 - cx
            chassis.leftmotor(50 - 0.4 * error)
            chassis.rightmotor(50 + 0.4 * error)

        else:
            error = 160 - cx
            chassis.leftmotor(25 - 0.7 * error)
            chassis.rightmotor(25 + 0.7 * error)

```

```

        else:
            if cx > 230:
                chassis.leftmotor(40)
                chassis.rightmotor(0)
            elif cx < 90:
                chassis.leftmotor(0)
                chassis.rightmotor(40)

    else:
        if sm.get_elapsed_time() > random.randint(15,20):
            return sm.switch("GoAway")

        chassis.leftmotor(-70)
        chassis.rightmotor(70)
        return

```

4) PickState — pick up the ball (picking up the ball)

```
class PickState(State):
    def on_tick(self, sm: StateMachine, *args, **kwargs):
        if sm.get_elapsed_time() <= 0.2:
            chassis.leftmotor(-40)
            chassis.rightmotor(-40)

        elif sm.get_elapsed_time() <= 1.0:
            pickservo.value = 1
            clipservo.value = 1
            chassis.leftmotor(40)
            chassis.rightmotor(40)
        elif sm.get_elapsed_time() <= 1.1:
            clipservo.value = 0
        elif sm.get_elapsed_time() <= 1.9:

            chassis.leftmotor(-20)
            chassis.rightmotor(-20)
        elif sm.get_elapsed_time() <= 2.4:
            pickservo.value = 0
        elif sm.get_elapsed_time() <= 2.9:
            chassis.leftmotor(-80)
            chassis.rightmotor(-80)
        else:
            chassis.leftmotor(0)
            chassis.rightmotor(0)
        return sm.switch("FindBall")
```

5) FindPutState — Find the red/green put area

```
class FindPutState(State):
    def __init__(self, color = None):
        self.color = color
        self.have = False

    def on_tick(self, sm: StateMachine, ballframe, *args, **kwargs):
        global putted
        if sm.get_elapsed_time() > 0.6:
            xs = []
            ws = []
            x = 320
            if self.color == 1:
                #green
                lower = np.array([0, 40, 0])
                upper = np.array([90, 255, 50])
            elif self.color == 0:
                #red
                lower = np.array([0, 0, 70])
                upper = np.array([90, 90, 255])
            mask = cv2.inRange(ballframe, lower, upper)
            #mask = cv2.dilate(mask, None, iterations=2)
            # mask = cv2.erode(mask, None, iterations=5)
            # mask = cv2.dilate(mask, None, iterations=12)
            cv2.imshow("mask", mask)
            contours, _ = cv2.findContours(mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
```

```

self.have = False
for contour in contours:
    x, y, w, h = cv2.boundingRect(contour)
    xs.append(x)
    ws.append(w)
    #print(x,y,w,h)
    if w > 40 and h > 30:
        self.have = True
        break

```

```

if self.have:
    x = xs[ws.index(max(ws))]
    #ballframe = cv2.rectangle(ballframe, (x,y), (x+w,y+h), (255,0,0), 2)
    min_x = int(x + w/2)
    if (y > 85):
        if self.color == 1:
            putted = 2
        elif self.color == 0:
            putted = 3
        return sm.switch("Put")
    elif min_x < 110:
        chassis.leftmotor(-30)
        chassis.rightmotor(40)
    elif min_x > 210:
        chassis.leftmotor(40)
        chassis.rightmotor(-30)
    else:
        chassis.leftmotor(70)
        chassis.rightmotor(70)
else:
    chassis.leftmotor(90)
    chassis.rightmotor(-90)

```

6) PutState — put action

```

class PutState(State):
    def on_tick(self, sm: StateMachine, *args, **kwargs):
        if sm.get_elapsed_time() <= 2:
            chassis.leftmotor(100)
            chassis.rightmotor(100)
        elif sm.get_elapsed_time() <= 3:
            chassis.leftmotor(-50)
            chassis.rightmotor(-50)
        elif sm.get_elapsed_time() <= 5.5:
            chassis.leftmotor(-80)
            chassis.rightmotor(80)
        elif sm.get_elapsed_time() <= 8.5:
            chassis.leftmotor(-100)
            chassis.rightmotor(-100)
        elif sm.get_elapsed_time() <= 9.5:
            chassis.leftmotor(0)
            chassis.rightmotor(0)
            backservo.value = 1

```

```

elif sm.get_elapsed_time() <= 9.3:
    chassis.leftmotor(50)
    chassis.rightmotor(50)
elif sm.get_elapsed_time() <= 9.6:
    chassis.leftmotor(-50)
    chassis.rightmotor(-50)
elif sm.get_elapsed_time() <= 9.9:
    chassis.leftmotor(50)
    chassis.rightmotor(50)
elif sm.get_elapsed_time() <= 10.2:
    chassis.leftmotor(-50)
    chassis.rightmotor(-50)
elif sm.get_elapsed_time() <= 11.5:
    backservo.value = 0
    chassis.leftmotor(0)
    chassis.rightmotor(0)
elif sm.get_elapsed_time() <= 13:
    chassis.leftmotor(100)
    chassis.rightmotor(100)
else:
    chassis.leftmotor(0)
    chassis.rightmotor(0)
    return sm.switch("FindBall")

```

This set of code handles the robot's complete ball-handling task: it uses a YOLO model to detect balls, navigates toward them, grabs the ball with servos when close enough, then searches for the corresponding red or green drop-off zone based on color. After reaching the zone, it releases the ball using the rear servo and returns to search for more balls.

3.Planning

- complete the program of entering and leaving the room.

Log number: 11

Event date: 5th of April,2026

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

- complete the program of entering and leaving the room.

2. Program

1) Entering the room after detected the silver line

```
silverframe = cv2.resize(frame,(224,224))
lineresults = linemodel.predict(silverframe, imgsz = 224)

print(sum(sum(Blackline)))
if sum(sum(Blackline)) < 1000:
    linestate = "white"
    print("white")
elif lineresults[0].probs.top1 == 1:
    linestate = "silver"
    print("silver")
elif lineresults[0].probs.top1 == 0:
    linestate = "black"
    print("black")
#print(has_left,has_right)
#print(self.goingdown)
if linestate == "silver":
    if putted == 3:
        return sm.switch("FuckGo")
    else:
        return sm.switch("CheckSilver")
```

2) Leaving the room

```
class GoState(State):
    def on_tick(self, sm: StateMachine, frame, Blackline, *args, **kwargs):
        #if sm.get_elapsed_time() > 0.5:
        if L.value == 1 or R.value == 1:
            return sm.switch("Turn")
        if dissensor.distance * 100 > 35:
            return sm.switch("Check")
        elif dissensor.distance * 100 < 4:
            return sm.switch("MoveL")
        elif dissensor.distance * 100 > 14:
            return sm.switch("MoveR")
        else:
            chassis.leftmotor(65)
            chassis.rightmotor(65)
            if L.value == 1 or R.value == 1:
                return sm.switch("Turn")
            silverframe = cv2.resize(frame, (224, 224))
            linerresults = linemodel.predict(silverframe, imgsz=224)
```

```
        if sum(sum(Blackline)) < 3000:
            linestate = "white"
            print("white")
        elif linerresults[0].probs.top1 == 1:
            linestate = "silver"
            print("silver")
        elif linerresults[0].probs.top1 == 0:
            linestate = "black"
            print("black")

        if linestate == "black":
            return sm.switch("GoGoGo")
        elif linestate == "silver":
            return sm.switch("GoBack")
```

TurnState (hit the wall and turn):

```
class TurnState(State):
    def on_tick(self, sm: StateMachine, frame, *args, **kwargs):

        if sm.get_elapsed_time() < 0.6:
            chassis.leftmotor(-80)
            chassis.rightmotor(80)
        else:
            return sm.switch("Go")
```

CheckState (check before entering the room):

```
class CheckState(State):
    def on_tick(self, sm: StateMachine, frame, *args, **kwargs):
        if sm.get_elapsed_time() < 0.7:
            chassis.leftmotor(80)
            chassis.rightmotor(80)
        elif sm.get_elapsed_time() < 2.1:
            chassis.leftmotor(80)
            chassis.rightmotor(-80)
        elif sm.get_elapsed_time() < 4.3:
            chassis.leftmotor(40)
            chassis.rightmotor(40)
        elif sm.get_elapsed_time() < 5:
            chassis.leftmotor(0)
            chassis.rightmotor(0)

        silverframe = cv2.resize(frame, (224, 224))
        results = linemodel.predict(silverframe, imgsz=224)
        self.line = results[0].probs.top1
        if self.line == 0:
            return sm.switch("GoGoGo")
        elif self.line == 2 or self.line == 1:
            return sm.switch("GoBack")
```

MoveLState / MoveRState (adjust position in the room):

```
class MoveLState(State):
    def on_tick(self, sm, *args, **kwargs):
        if sm.get_elapsed_time() < 0.6:
            chassis.leftmotor(0)
            chassis.rightmotor(-80)
        elif sm.get_elapsed_time() < 1.5:
            chassis.leftmotor(-80)
            chassis.rightmotor(0)
        elif sm.get_elapsed_time() < 2.3:
            chassis.leftmotor(80)
            chassis.rightmotor(80)
        else:
            chassis.stop()
            return sm.switch("Go")
```

```
class MoveRState(State):
    def on_tick(self, sm, *args, **kwargs):
        if sm.get_elapsed_time() < 0.6:
            chassis.leftmotor(-80)
            chassis.rightmotor(0)
        elif sm.get_elapsed_time() < 1.5:
            chassis.leftmotor(0)
            chassis.rightmotor(-80)
        elif sm.get_elapsed_time() < 2.3:
            chassis.leftmotor(80)
            chassis.rightmotor(80)
        else:
            chassis.stop()
            return sm.switch("Go")
```

GoGoGo (rush out after entering the room):

```
class GoGoGo(State):
    def on_tick(self, sm, *args, **kwargs):
        if sm.get_elapsed_time() < 0.4:
            chassis.leftmotor(50)
            chassis.rightmotor(50)
            if L.value == 1:
                return sm.switch("LB")
            if R.value == 1:
                return sm.switch("RB")
        else:
            return sm.switch("Line_Follow")
```

BeforeGoState (prepare before entering the room):

```

class BeforeGoState(State):
    def on_tick(self, sm, *args, **kwargs):
        if sm.get_elapsed_time() < 1.5:
            chassis.leftmotor(-100)
            chassis.rightmotor(-100)
        elif sm.get_elapsed_time() < 2.6:
            chassis.leftmotor(80)
            chassis.rightmotor(-80)
        else:
            chassis.stop()
            return sm.switch("Go")

```

GoBackState (Leave the room: Back + Turn):

```

class GoBackState(State):
    def on_tick(self, sm, *args, **kwargs):
        if sm.get_elapsed_time() < 1.1:
            chassis.leftmotor(-80)
            chassis.rightmotor(-80)
        elif sm.get_elapsed_time() < 2.3:
            chassis.leftmotor(-80)
            chassis.rightmotor(80)
        elif sm.get_elapsed_time() < 3:
            chassis.leftmotor(80)
            chassis.rightmotor(80)
        else:
            return sm.switch("Go")

```

This group of states controls the robot's room entry and exit. The GoState uses distance sensors and line detection (black/silver) to navigate inside the room, adjust position, avoid walls, exit when seeing a black line, and back out when seeing a silver line.

3.Planning

- complete the program for the finishing red line

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

- complete the program for the finishing red line

2. Program

```
class RedStopState(State):
    def on_tick(self, sm: StateMachine, *args, **kwargs):
        if sm.get_elapsed_time() <= 12:
            chassis.leftmotor(0)
            chassis.rightmotor(0)
        else:
            return sm.switch("Line_Follow")
```

3. Planning

- test the entire robot and improve it

Members attended: Li Cheng Lok, Lloyd, Mike.V, Wong loi Cheng

Editor: Lloyd

Location: Macau Pui Ching Secondary School Innovation Science and Education Center

1. Tasks done today

- test the entire robot and improve it

2. Problem

When clamping the ball, the accuracy will be significantly affected due to light leakage from the environment, which may cause unstable visual detection, misalignment of the ball position, and inconsistent clamping movements, ultimately reducing the success rate of picking up the ball.

3. Solution

Therefore, to solve this problem and improve the stability and success rate of ball clamping, we specially added light-blocking baffles to the bottom of the robot.

